

Data Structures and Algorithms

GÉZA HORVÁTH

Fifth Lecture

Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 Elementary data structures: stacks, queues, lists.
- 5 **Tables, hash tables, hash functions, collisions.**
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 Red-black trees.
- 9 B-trees.
- 10 Graphs, traversal, shortest path.
- 11 Parallel algorithms: matrix multiplication, merge sort.
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

The table – abstract data type

In computer science, a table is a collection of data elements, whose complex elements contain

- **key**, which must be unique and
- **value**, which can be complex itself.

The elements are identified by their unique keys.

The table – data structure

Table properties:

- solid
- associative
- dynamic
- sometimes continuously represented, sometimes represented by pointers

Possible tables

There are many possibility for tables:

- standard table
- self-ordering table
- hash table

The table – operations

- INSERT – adds elements to the table.
- DELETE – removes elements from the table.
- SEARCH – finds elements in the table.

The table – data structure operations

Implemented data structure operations:

- **create** data structure – **linked or countinuous** representation
- **modify** data structure
 - add elements – INSERT
 - delete elements – DELETE
 - change elements – **not important**
- **access** – SEARCH

The table – possibilities

	sorted	disordered
continous	yes	yes
linked	yes	yes

Self-ordering table

- Self-ordering table is effective if the **accessing frequency** of the elements are highly different.
- The only operation which is different to the original one is the **SEARCH**. After using **SEARCH**, we move the element we found to the first position. As a result, the more frequently used elements will be at the beginning of the table.
- We can use **linked representation** and **serial search**, because it is easy to move the elements to the front, and serial search is effective if the more frequently used elements are at the beginning of the table.
- Self-ordering table **reacts** very fast if the accessing frequency of the elements changes.

Self-ordering table – example

Shop in the morning:

200	coffee
300	sandwich
350	cola
150	tea
1.120	cigarette
400	cake
5.200	whisky
290	beer
4.800	steak
14.500.000	BMW

Self-ordering table – example

Shop at noon:

4.800	steak
200	coffee
350	cola
400	cake
290	beer
300	sandwich
150	tea
1.120	cigarette
5.200	whisky
14.500.000	BMW

Self-ordering table – example

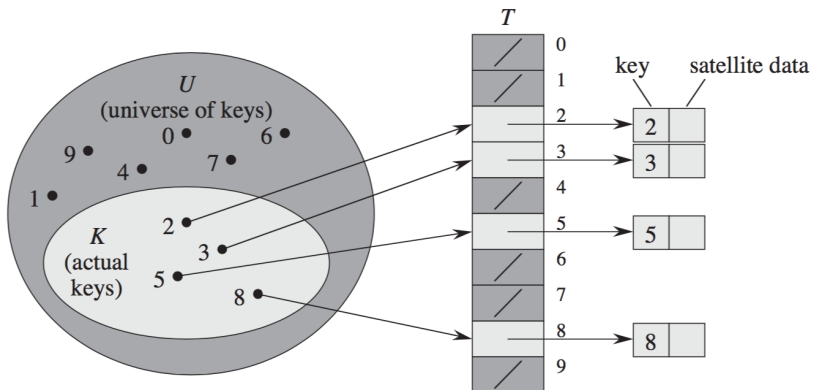
Shop in the evening:

290	beer
1.120	cigarette
5.200	whisky
4.800	steak
350	cola
300	sandwich
400	cake
150	tea
200	coffee
14.500.000	BMW

Direct-address table

- Direct addressing is a simple technique that works well for tables when the universe U of keys is reasonably small.
- Suppose that an application needs a table in which each element has a unique key drawn from the universe $U = \{0, 1, \dots, m-1\}$, where m is not too large.
- To represent the table, we use an array, or direct-address table, denoted by $T[0 \dots m-1]$, in which each position, or slot, corresponds to a key in the universe U .

Direct-address table



Direct-address table – operations

DIRECT-ADDRESS-SEARCH(T, k)

1 **return** $T[k]$

DIRECT-ADDRESS-INSERT(T, x)

1 $T[x.key] = x$

DIRECT-ADDRESS-DELETE(T, x)

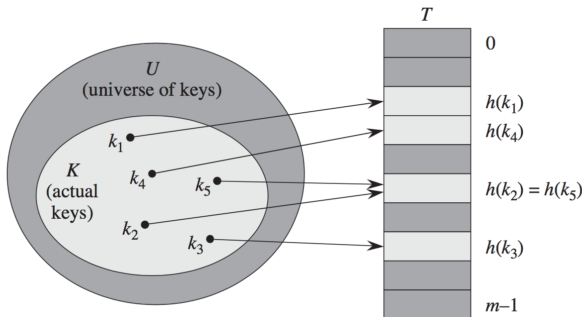
1 $T[x.key] = \text{NIL}$

Hash table

- The downside of direct addressing is obvious: if the universe U is large, storing a table T of size $|U|$ may be impractical, or even impossible, given the memory available on a typical computer. Furthermore, the set K of keys actually stored may be so small relative to U that most of the space allocated for T would be wasted.
- With direct addressing, an element with key k is stored in slot k . With hashing, this element is stored in slot $h(k)$; that is, we use a hash function h to compute the slot from the key k .

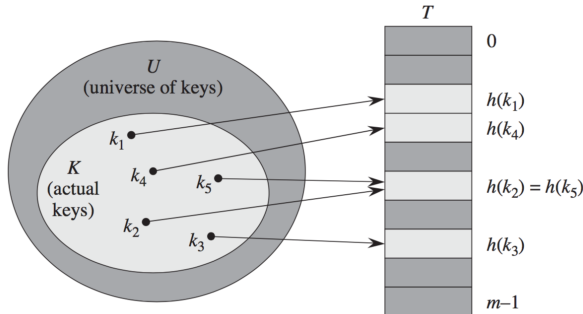
Hash table

- Hash function h maps the universe U of keys into the slots of a hash table $T[0 \dots m-1]$.
- $h: U \rightarrow \{0, 1, \dots, m-1\}$, where the size m of the hash table is typically much less than $|U|$.



Hash table – collision

- There is one hitch: two keys may hash to the same slot. We call this situation a **collision**.
- In this example, keys k_2 and k_5 have the same **hash value**.
- Fortunately, we have effective techniques for resolving the conflict created by collisions.

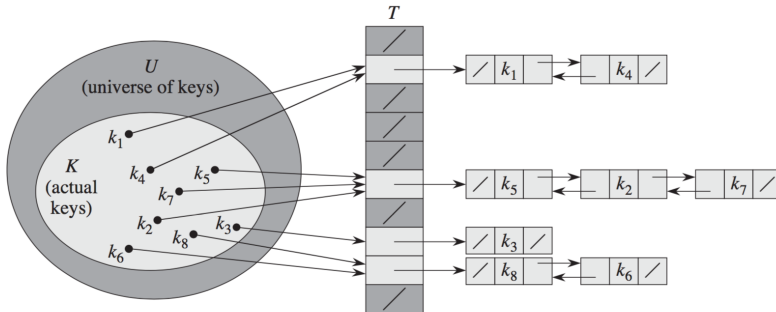


Hash function – the division method

- In the **division method** for creating hash functions, we map a key k into one of m slots by taking the remainder of k divided by m .
- The hash function is $h(k) = k \bmod m$.
- For example, if the hash table has size $m=12$, then the possible hash values are between 0 and 11.
- If the hash table has size $m=12$ and the key is $k=100$, then $h(k)=4$.

Collision resolution by chaining

- In chaining, we place all the elements that hash to the same slot into the same linked list.
- Slot j contains a pointer to the head of the list of all stored elements that hash to j ; if there are no such elements, slot j contains NIL.



Collision resolution by chaining

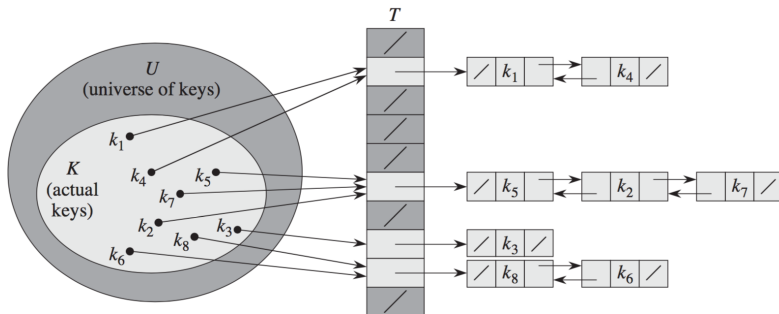


Figure 11.3 Collision resolution by chaining. Each hash-table slot $T[j]$ contains a linked list of all the keys whose hash value is j . For example, $h(k_1) = h(k_4)$ and $h(k_5) = h(k_7) = h(k_2)$. The linked list can be either singly or doubly linked; we show it as doubly linked because deletion is faster that way.

Collision resolution by chaining – operations

CHAINED-HASH-INSERT(T, x)

1 insert x at the head of list $T[h(x.key)]$

CHAINED-HASH-SEARCH(T, k)

1 search for an element with key k in list $T[h(k)]$

CHAINED-HASH-DELETE(T, x)

1 delete x from the list $T[h(x.key)]$

Collision resolution by open addressing

- In open addressing, all elements occupy the hash table itself. That is, each table entry contains either an element or NIL.
- When searching for an element, we systematically examine table slots until either we find the desired element or we have ascertained that the element is not in the table.
- No lists and no elements are stored outside the table, unlike in chaining.
- The advantage of open addressing is that it avoids pointers altogether.

Open addressing – operations

HASH-INSERT(T, k)

```
1   $i = 0$ 
2  repeat
3       $j = h(k, i)$ 
4      if  $T[j] == \text{NIL}$ 
5           $T[j] = k$ 
6          return  $j$ 
7      else  $i = i + 1$ 
8  until  $i == m$ 
9  error “hash table overflow”
```


Open addressing – operations

HASH-SEARCH(T, k)

1 $i = 0$

2 **repeat**

3 $j = h(k, i)$

4 **if** $T[j] == k$

5 **return** j

6 $i = i + 1$

7 **until** $T[j] == \text{NIL}$ or $i == m$

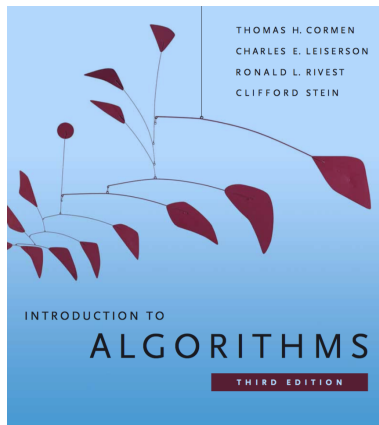
8 **return** NIL

Open addressing – operations

Deletion from an open-address hash table is difficult. When we delete a key from slot i , we cannot simply mark that slot as empty by storing NIL in it. If we did, we might be unable to retrieve any key k during whose insertion we had probed slot i and found it occupied. We can solve this problem by marking the slot, storing in it the special value DELETED instead of NIL. We would then modify the procedure HASH-INSERT to treat such a slot as if it were empty so that we can insert a new key there. We do not need to modify HASH-SEARCH, since it will pass over DELETED values while searching. When we use the special value DELETED, however, search times no longer depend on the load factor α , and for this reason chaining is more commonly selected as a collision resolution technique when keys must be deleted.

Details will be explained in practical class with examples.

References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.